

index.html — estrutura DOM da árvore 2D

```
<section id="tree-view" class="tree-view hidden">

  <div class="tree-toolbar">

    <button id="btn-back" class="btn-primary" type="button">

      <span></span> Voltar aos Projetos

    </button>

    <div class="tree-info">

      <span id="tree-project-name" class="tree-project-name"></span>

      <span id="tree-node-count" class="tree-node-count"></span>

    </div>

    <a id="tree-demo-link" href="#" target="_blank" rel="noopener" class="btn-ghost hidden">▶ Abrir Demo</a>
    <a id="tree-galaxy-link" href="#" class="btn-ghost hidden">🌌 Modo Galaxy 3D</a>
    <a id="tree-repo-link" href="#" target="_blank" rel="noopener" class="btn-ghost">Ver Repositório</a>

  </div>

  <div class="tree-scene-wrap" id="tree-scene-wrap">
    <div class="tree-fs-bar hidden" id="tree-fs-bar" aria-hidden="true">
      <span id="tree-fs-name" class="tree-fs-name"></span>
      <button type="button" id="btn-tree-fs-exit" class="tree-fs-exit">Sair da tela cheia ⏏</button>
    </div>
    <div class="tree-zoom" id="tree-zoom" role="group" aria-label="Zoom da árvore">
      <button type="button" id="btn-zoom-out" class="tree-zoom-btn" aria-label="Diminuir zoom" title="Diminuir zoom">-</button>
      <span id="tree-zoom-label" class="tree-zoom-label">35%</span>
      <button type="button" id="btn-zoom-in" class="tree-zoom-btn" aria-label="Aumentar zoom" title="Aumentar zoom">+</button>
      <button type="button" id="btn-zoom-reset" class="tree-zoom-btn tree-zoom-reset" aria-label="Visão geral" title="Visão geral (35%)">🔄</button>
      <button type="button" id="btn-tree-fullscreen" class="tree-zoom-btn tree-zoom-fs" aria-label="Tela cheia da árvore" title="Tela cheia da árvore">⏏</button>
    </div>

    <div id="tree-scene" class="tree-scene">
      <div id="tree-zoom-space" class="tree-zoom-space">
        <div id="tree-stage" class="tree-stage">
          <svg id="tree-lines" class="tree-lines" aria-hidden="true"></svg>
          <div id="tree-nodes" class="tree-nodes"></div>
        </div>
      </div>
    </div>

  </div>

  <p class="tree-hint">Abre em 35% para visão geral • Scroll na árvore • zoom até 100% • 🔄 panorama • ⏏ tela cheia</p>

</section>
```

js/tree-renderer.js

```
/* Tree Renderer - layout radial estável (inclui árvores grandes 100-400+ nós) */
const TreeRenderer = (() => {
  const SIBLING_PAD = 1.42;
  const PARENT_PAD = 1.22;
  const STAGE_PAD = 80;
  const SEPARATION_ITERS = 56;
  const FILL_RATIO = 0.88;
  /** Evita órbita explosiva quando subárvores profundas têm muitos irmãos. */
  const MAX_CHILD_R_FOR_ORBIT = 420;
  const MAX_SUB_R = 2400;
  const MAX_ROOT_ORBIT = 920;

  function buildTree(nodes) {
    const map = new Map();
    nodes.forEach(n => map.set(n.id, { ...n, children: [] }));
    const root = nodes.find(n => !n.parent);
    nodes.forEach(n => {
      if (n.parent && map.has(n.parent)) map.get(n.parent).children.push(map.get(n.id));
    });
  }
})();
```

```

    return root ? map.get(root.id) : null;
}

function countNodes(node) {
    let c = 1;
    (node.children || []).forEach(ch => { c += countNodes(ch); });
    return c;
}

function nodeHalfSize(layer, dense) {
    const sizes = dense
        ? { root: 40, module: 26, file: 22, function: 20 }
        : { root: 50, module: 36, file: 29, function: 25 };
    return sizes[layer] || sizes.function;
}

function childSpread(count, isRoot) {
    if (count <= 1) return 0;
    if (isRoot) return Math.PI * 2;
    return Math.min(Math.PI * 1.25, 0.55 + count * 0.48);
}

function orbitFromChildren(half, childRs, isRoot, mega) {
    const n = childRs.length;
    if (n === 0) return 0;
    const capped = childRs.map((r) => Math.min(r, mega ? MAX_CHILD_R_FOR_ORBIT * 0.72 : MAX_CHILD_R_FOR_ORBIT));
    if (n === 1) return (half + capped[0]) * PARENT_PAD;

    // Mega-árvores: raio ~ sqrt(n) – evita (a+b)/sin(step) explodir
    if (mega) {
        const maxC = Math.max(...capped);
        return half + maxC * 1.12 + Math.sqrt(n) * 38 + Math.min(n, 24) * 6;
    }

    const spread = childSpread(n, isRoot);
    const step = isRoot ? spread / n : spread / (n - 1);
    let orbit = 0;
    for (let i = 0; i < n; i++) {
        const a = capped[i];
        const b = capped[isRoot ? (i + 1) % n : Math.min(i + 1, n - 1)];
        if (!isRoot && i === n - 1) break;
        const need = ((a + b) * SIBLING_PAD) / (2 * Math.sin(Math.max(step, 0.12) / 2));
        orbit = Math.max(orbit, need);
    }
    orbit = Math.max(orbit, (half + Math.max(...capped)) * PARENT_PAD);
    return orbit;
}

function measureSubtree(node, dense, isRoot, minRootOrbit, mega) {
    const half = nodeHalfSize(node.layer || 'function', dense);
    const children = node.children || [];
    if (!children.length) {
        node._subR = half;
        node._orbit = 0;
        return half;
    }

    children.forEach(ch => measureSubtree(ch, dense, false, 0, mega));
    const childRs = children.map(ch => ch._subR);
    let orbit = orbitFromChildren(half, childRs, isRoot, mega);

    if (isRoot && minRootOrbit) {
        orbit = Math.max(orbit, minRootOrbit);
    }

    if (children.length >= 5) {
        orbit *= mega ? 1.04 : 1.08;
    }

    if (isRoot) {
        orbit = Math.min(orbit, MAX_ROOT_ORBIT);
    }

    node._orbit = orbit;
    node._subR = orbit + Math.max(...childRs.map((r) => Math.min(r, mega ? MAX_CHILD_R_FOR_ORBIT : r)));
    if (!isRoot && node._subR > MAX_SUB_R) {
        const scale = MAX_SUB_R / node._subR;
        node._orbit *= scale;
        node._subR = MAX_SUB_R;
    }
}

```

```

    }
    return node._subR;
}

function layoutNode(node, angle, depth, positions, cx, cy) {
    positions.set(node.id, { x: cx, y: cy, node, depth, angle });
    const children = node.children || [];
    if (!children.length) return;

    const orbitR = node._orbit || 0;
    const n = children.length;
    const isRoot = depth === 0;

    if (n === 1) {
        const a = isRoot ? -Math.PI / 2 : angle;
        layoutNode(
            children[0], a, depth + 1, positions,
            cx + Math.cos(a) * orbitR,
            cy + Math.sin(a) * orbitR
        );
        return;
    }

    const spread = childSpread(n, isRoot);
    const step = isRoot ? spread / n : spread / (n - 1);
    const start = isRoot ? -Math.PI / 2 : angle - spread / 2;

    children.forEach((child, i) => {
        const a = start + step * i;
        layoutNode(
            child, a, depth + 1, positions,
            cx + Math.cos(a) * orbitR,
            cy + Math.sin(a) * orbitR
        );
    });
}

function resolveOverlaps(positions, dense) {
    const items = Array.from(positions.values());
    const iters = items.length > 200 ? 28 : SEPARATION_ITERS;
    for (let iter = 0; iter < iters; iter++) {
        let moved = false;
        for (let i = 0; i < items.length; i++) {
            for (let j = i + 1; j < items.length; j++) {
                const a = items[i];
                const b = items[j];
                const ra = nodeHalfSize(a.node.layer || 'function', dense);
                const rb = nodeHalfSize(b.node.layer || 'function', dense);
                const minDist = (ra + rb) * SIBLING_PAD;
                let dx = b.x - a.x;
                let dy = b.y - a.y;
                let dist = Math.hypot(dx, dy);
                if (dist < 0.01) {
                    dx = (Math.random() - 0.5) || 0.01;
                    dy = (Math.random() - 0.5) || 0.01;
                    dist = Math.hypot(dx, dy);
                }
                if (dist >= minDist) continue;

                const push = (minDist - dist) / 2;
                const ux = dx / dist;
                const uy = dy / dist;
                const aRoot = a.node.layer === 'root';
                const bRoot = b.node.layer === 'root';

                if (!aRoot) {
                    a.x -= ux * push;
                    a.y -= uy * push;
                    moved = true;
                }
                if (!bRoot) {
                    b.x += ux * (aRoot ? push * 2 : push);
                    b.y += uy * (aRoot ? push * 2 : push);
                    moved = true;
                } else if (aRoot) {
                    a.x -= ux * push * 2;
                    a.y -= uy * push * 2;
                    moved = true;
                }
            }
        }
    }
}

```

```

    }
  }
  if (!moved) break;
}
}

function boundsOf(positions, dense) {
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  positions.forEach((pos) => {
    const margin = nodeHalfSize(pos.node.layer || 'function', dense) * 1.3;
    minX = Math.min(minX, pos.x - margin);
    minY = Math.min(minY, pos.y - margin);
    maxX = Math.max(maxX, pos.x + margin);
    maxY = Math.max(maxY, pos.y + margin);
  });
  return { minX, minY, maxX, maxY, width: maxX - minX, height: maxY - minY };
}

function normalizeStage(stage, positions, dense, sceneW, sceneH) {
  let b = boundsOf(positions, dense);

  const targetW = Math.max(sceneW * FILL_RATIO, 320);
  const targetH = Math.max(sceneH * FILL_RATIO, 280);
  const scaleUp = Math.min(targetW / Math.max(b.width, 1), targetH / Math.max(b.height, 1));

  if (scaleUp > 1.02) {
    const cx = (b.minX + b.maxX) / 2;
    const cy = (b.minY + b.maxY) / 2;
    positions.forEach((pos) => {
      pos.x = cx + (pos.x - cx) * scaleUp;
      pos.y = cy + (pos.y - cy) * scaleUp;
    });
    b = boundsOf(positions, dense);
  }

  const MAX_STAGE = 14000;
  if (b.width > MAX_STAGE || b.height > MAX_STAGE) {
    const shrink = Math.min(MAX_STAGE / Math.max(b.width, 1), MAX_STAGE / Math.max(b.height, 1));
    const cx = (b.minX + b.maxX) / 2;
    const cy = (b.minY + b.maxY) / 2;
    positions.forEach((pos) => {
      pos.x = cx + (pos.x - cx) * shrink;
      pos.y = cy + (pos.y - cy) * shrink;
    });
    b = boundsOf(positions, dense);
  }

  const contentW = b.width + STAGE_PAD * 2;
  const contentH = b.height + STAGE_PAD * 2;
  const width = Math.max(contentW, sceneW);
  const height = Math.max(contentH, sceneH);

  const offsetX = (width - b.width) / 2 - b.minX;
  const offsetY = (height - b.height) / 2 - b.minY;
  positions.forEach((pos) => {
    pos.x += offsetX;
    pos.y += offsetY;
  });

  stage.style.width = width + 'px';
  stage.style.height = height + 'px';
  stage.style.transform = 'none';
  stage.style.transformOrigin = '0 0';
  stage.dataset.baseWidth = String(width);
  stage.dataset.baseHeight = String(height);

  return { width, height };
}

function render(container, linesEl, nodes, projectColor, projectSlug) {
  container.innerHTML = '';
  linesEl.innerHTML = '';
  const tree = buildTree(nodes);
  if (!tree) return;

  const stage = container.parentElement;
  const zoomSpace = stage.parentElement;
  const scene = zoomSpace.parentElement;
  stage.style.transform = '';

```

```

const sceneW = Math.max(scene.clientWidth || 1000, 640);
const sceneH = Math.max(scene.clientHeight || 640, 420);
const nodeCount = countNodes(tree);
const dense = nodeCount > 14;
const mega = nodeCount >= 80;

const minRootOrbit = Math.min(
  MAX_ROOT_ORBIT * 0.85,
  Math.min(sceneW, sceneH) * (mega ? 0.42 : nodeCount >= 40 ? 0.30 : 0.34),
);
measureSubtree(tree, dense, true, minRootOrbit, mega);

const positions = new Map();
layoutNode(tree, -Math.PI / 2, 0, positions, 0, 0);
resolveOverlaps(positions, dense);
const stageSize = normalizeStage(stage, positions, dense, sceneW, sceneH);

linesEl.setAttribute('width', stageSize.width);
linesEl.setAttribute('height', stageSize.height);
linesEl.setAttribute('viewBox', '0 0 ' + stageSize.width + ' ' + stageSize.height);

positions.forEach((pos) => {
  const n = pos.node;
  if (!n.parent) return;
  const parent = positions.get(n.parent);
  if (!parent) return;
  const line = document.createElementNS('http://www.w3.org/2000/svg', 'line');
  line.setAttribute('x1', parent.x);
  line.setAttribute('y1', parent.y);
  line.setAttribute('x2', pos.x);
  line.setAttribute('y2', pos.y);
  line.setAttribute('stroke', projectColor || '#38bdf8');
  line.setAttribute('stroke-opacity', '0.35');
  line.setAttribute('stroke-width', '1.5');
  linesEl.appendChild(line);
});

positions.forEach((pos) => {
  const n = pos.node;
  const isRoot = n.layer === 'root';
  const el = document.createElement(isRoot ? 'div' : 'a');
  el.className = 'tree-node' + (dense ? ' tree-node-dense' : '');
  el.dataset.id = n.id;
  el.dataset.layer = n.layer || 'function';
  el.style.left = pos.x + 'px';
  el.style.top = pos.y + 'px';
  el.style.zIndex = isRoot ? 80 : n.layer === 'module' ? 50 : n.layer === 'file' ? 30 : 10;

  if (!isRoot) {
    el.href = AppConfig.nodePageUrl(projectSlug, n.id);
    el.removeAttribute('title');
  } else {
    el.style.cursor = 'default';
  }

  const inner = document.createElement('span');
  inner.className = 'tree-node-inner';
  const maxLen = dense ? 12 : 15;
  const title = n.title || n.id;
  inner.textContent = title.length > maxLen ? title.slice(0, maxLen - 1) + '...' : title;
  el.appendChild(inner);
  container.appendChild(el);
});

return positions;
}

return { render, buildTree, countNodes };
})();

```

css/style.css — arvore 2D

```

.tree-scene-wrap {
  position: relative;
  flex: 1;
  min-height: 0;
  display: flex;

```

```

    flex-direction: column;
}

.tree-scene-wrap:fullscreen {
    background: #060810;
    padding: 0.65rem;
    box-sizing: border-box;
}

.tree-scene-wrap:-webkit-full-screen {
    background: #060810;
    padding: 0.65rem;
    box-sizing: border-box;
}

.tree-fs-bar {
    position: absolute;
    top: 14px;
    left: 14px;
    z-index: 45;
    display: flex;
    align-items: center;
    gap: 0.75rem;
    padding: 0.4rem 0.65rem;
    border-radius: 10px;
    background: rgba(12, 18, 32, 0.92);
    border: 1px solid var(--border);
    backdrop-filter: blur(10px);
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.35);
}

.tree-fs-name {
    font-size: 0.85rem;
    font-weight: 600;
    color: var(--text);
    max-width: min(42vw, 320px);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}

.tree-fs-exit {
    font-family: inherit;
    font-size: 0.75rem;
    padding: 0.35rem 0.55rem;
    border-radius: 6px;
    border: 1px solid var(--border);
    background: rgba(56, 189, 248, 0.12);
    color: var(--accent);
    cursor: pointer;
}

.tree-fs-exit:hover {
    background: rgba(56, 189, 248, 0.22);
}

.tree-scene-wrap:fullscreen .tree-scene,
.tree-scene-wrap:-webkit-full-screen .tree-scene {
    flex: 1;
    min-height: 0;
    border-radius: 12px;
}

.tree-scene {
    position: relative; width: 100%;
    flex: 1; min-height: 0;
    overflow: auto;
    border-radius: 16px; border: 1px solid var(--border);
    background: radial-gradient(ellipse at center, rgba(20,30,50,.9) 0%, rgba(6,8,15,.95) 100%);
    scrollbar-width: thin;
    scrollbar-color: rgba(56,189,248,.45) rgba(255,255,255,.06);
}

.tree-scene::-webkit-scrollbar { width: 10px; height: 10px; }
.tree-scene::-webkit-scrollbar-thumb {
    background: rgba(56,189,248,.45);
    border-radius: 999px;
}

.tree-scene::-webkit-scrollbar-track { background: rgba(255,255,255,.06); }
.tree-zoom {
    position: absolute;

```

```

top: 14px;
right: 14px;
z-index: 40;
display: flex;
align-items: center;
gap: .35rem;
padding: .35rem;
border-radius: 12px;
background: rgba(12, 18, 32, .92);
border: 1px solid var(--border);
box-shadow: 0 8px 24px rgba(0,0,0,.35);
backdrop-filter: blur(10px);
}

.tree-zoom-btn {
width: 36px; height: 36px;
border: 1px solid var(--border);
border-radius: 8px;
background: rgba(56,189,248,.1);
color: var(--text);
font-size: 1.25rem;
font-weight: 700;
line-height: 1;
cursor: pointer;
transition: background .15s, border-color .15s, transform .15s;
}

.tree-zoom-btn:hover {
background: rgba(56,189,248,.22);
border-color: var(--accent);
}

.tree-zoom-btn:active { transform: scale(.96); }
.tree-zoom-btn:disabled {
opacity: .4;
cursor: not-allowed;
transform: none;
}

.tree-zoom-reset {
font-size: 1rem;
}

.tree-zoom-fs {
font-size: 1rem;
}

.tree-zoom-label {
min-width: 3.4rem;
text-align: center;
font-family: var(--mono);
font-size: .75rem;
font-weight: 600;
color: var(--accent);
}

.tree-zoom-space {
position: relative;
min-width: 100%;
min-height: 100%;
}

.tree-stage {
position: relative;
transform-origin: 0 0;
will-change: transform, width, height;
}

.tree-lines {
position: absolute; inset: 0; width: 100%; height: 100%;
pointer-events: none;
}

.tree-lines line {
stroke: rgba(56,189,248,.25); stroke-width: 1.5;
filter: drop-shadow(0 0 4px rgba(56,189,248,.3));
}

.tree-nodes { position: absolute; inset: 0; pointer-events: none; }
.tree-nodes .tree-node { pointer-events: auto; }

/* Nodes */
.tree-node {
position: absolute;
transform: translate(-50%, -50%);
cursor: pointer;
transition: transform .2s, filter .2s;
z-index: 2;
text-decoration: none;
color: inherit;

```

```

    pointer-events: auto;
}
a.tree-node:hover { text-decoration: none; }
.tree-node-inner {
  display: flex; align-items: center; justify-content: center;
  border-radius: 50%;
  font-size: .68rem; font-weight: 600; text-align: center;
  color: #fff; line-height: 1.2; padding: 4px;
  word-break: break-word;
  transition: all .25s;
}
.tree-node[data-layer="root"] .tree-node-inner {
  width: 100px; height: 100px; font-size: .74rem;
  background: radial-gradient(circle at 30% 30%, #fde68a, var(--gold) 50%, #b45309);
  box-shadow: 0 0 30px rgba(251,191,36,.5), 0 0 60px rgba(251,191,36,.2);
  animation: pulse-root 3s ease-in-out infinite;
}
.tree-node[data-layer="module"] .tree-node-inner {
  width: 72px; height: 72px;
  background: radial-gradient(circle at 30% 30%, #7dd3fc, var(--accent) 60%, #0369a1);
  box-shadow: 0 0 20px rgba(56,189,248,.4);
}
.tree-node[data-layer="file"] .tree-node-inner {
  width: 58px; height: 58px;
  background: radial-gradient(circle at 30% 30%, #d8b4fe, var(--purple) 60%, #7c3aed);
  box-shadow: 0 0 16px rgba(192,132,252,.35);
}
.tree-node[data-layer="function"] .tree-node-inner {
  width: 50px; height: 50px;
  background: radial-gradient(circle at 30% 30%, #6ee7b7, var(--green) 60%, #059669);
  box-shadow: 0 0 14px rgba(52,211,153,.35);
}
.tree-node-dense[data-layer="module"] .tree-node-inner { width: 58px; height: 58px; font-size: .58rem; }
.tree-node-dense[data-layer="file"] .tree-node-inner { width: 50px; height: 50px; font-size: .55rem; }
.tree-node-dense[data-layer="function"] .tree-node-inner { width: 43px; height: 43px; font-size: .53rem; }
.tree-node-dense[data-layer="root"] .tree-node-inner { width: 86px; height: 86px; font-size: .66rem; }
.tree-node:hover { z-index: 10; }
.tree-node:hover .tree-node-inner {
  transform: scale(1.12);
  filter: brightness(1.15);
}
@keyframes pulse-root {
  0%, 100% { box-shadow: 0 0 30px rgba(251,191,36,.5), 0 0 60px rgba(251,191,36,.2); }
  50% { box-shadow: 0 0 40px rgba(251,191,36,.7), 0 0 80px rgba(251,191,36,.35); }
}

```

js/app.js — montagem da árvore 2D

```

/* App - galeria, árvore, busca e zoom */
(() => {
  const $ = (s) => document.querySelector(s);
  const gallery = $('#gallery');
  const treeView = $('#tree-view');
  const projectGrid = $('#project-grid');
  const treeNodes = $('#tree-nodes');
  const treeLines = $('#tree-lines');
  const treeScene = $('#tree-scene');
  const treeStage = $('#tree-stage');
  const treeZoomSpace = $('#tree-zoom-space');
  function renderTree() {
    requestAnimationFrame(() => {
      TreeRenderer.render(treeNodes, treeLines, currentProject.nodes, currentProject.color, currentProject.slug);
      if (fitOverviewOnRender) {
        zoom = computeOverviewZoom();
        fitOverviewOnRender = false;
      }
      applyZoom(zoom, false);
      centerTreeInView();
      bindNodeEvents();
    });
  }

  function openTree(project) {
    if (project.comingSoon) return;
    currentProject = project;
    nodeMap = new Map(project.nodes.map(n => [n.id, n]));
    zoom = ZOOM_INITIAL;
    fitOverviewOnRender = true;
  }
}

```

```

gallery.classList.add('hidden');
treeView.classList.remove('hidden');
history.replaceState(null, '', AppConfig.indexUrl('tree=' + encodeURIComponent(project.slug)));
$('#tree-project-name').textContent = project.name;
$('#tree-node-count').textContent = project.nodes.length + ' nós de código';
$('#tree-repo-link').href = project.repoUrl;
const demoLink = $('#tree-demo-link');
if (project.demoUrl) {
  demoLink.href = project.demoUrl;
  demoLink.classList.remove('hidden');
} else {
  demoLink.removeAttribute('href');
  demoLink.classList.add('hidden');
}
const galaxyLink = $('#tree-galaxy-link');
if (galaxyLink) {
  galaxyLink.href = 'galaxy/index.html?project=' + encodeURIComponent(project.slug);
  galaxyLink.classList.remove('hidden');
}
renderTree();
}

```

scripts/export-universe-graph.py — grafo 3D

```

def layer_size(layer):
    return {"root": 2.2, "module": 1.35, "file": 0.85, "function": 0.62}.get(layer, 0.6)

def export_project(project):
    slug = project.get("slug")
    if not slug:
        return None
    nodes_in = project.get("nodes", [])
    if not nodes_in:
        return None

    nodes = []
    links = []
    used_categories = set()

    for n in nodes_in:
        cat = category_for(n, slug)
        used_categories.add(cat)
        nodes.append({
            "id": n["id"],
            "label": n.get("title") or n["id"],
            "parent": n.get("parent"),
            "layer": n.get("layer", "file"),
            "category": cat,
            "categoryLabel": CATEGORIES[cat][0],
            "color": CATEGORIES[cat][1],
            "size": layer_size(n.get("layer", "file")),
            "file": n.get("file", ""),
            "description": n.get("description", ""),
        })
        if n.get("parent"):
            links.append({"source": n["parent"], "target": n["id"], "type": "dependency"})

    return {
        "slug": slug,
        "name": project.get("name", slug),
        "repoUrl": project.get("repoUrl", ""),
        "color": project.get("color", "#06b6d4"),
        "demoUrl": project.get("demoUrl", ""),
        "nodeCount": len(nodes),
        "categories": {k: {"label": v[0], "color": v[1]} for k, v in CATEGORIES.items() if k in used_categories or k == "other"},
        "nodes": nodes,
        "links": links,
    }

```

universe/src/utlis/layout.js

```

/** Layout hierárquico radial 3D — espelha a lógica da árvore 2D (raios alinhados aos nós). */
export function computeLayout(graph) {
  const { nodes } = graph;
  const children = new Map();

```

```

nodes.forEach((n) => {
  if (n.parent) {
    if (!children.has(n.parent)) children.set(n.parent, []);
    children.get(n.parent).push(n.id);
  }
});

children.forEach((ids) => ids.sort());

const root =
  nodes.find((n) => n.layer === "root") ||
  nodes.find((n) => !n.parent) ||
  nodes[0];

const positions = new Map();
positions.set(root.id, { x: 0, y: 0, z: 0 });

const level1 = children.get(root.id) || [];
// Anel no plano XZ – mesmo espírito do 2D
const R1 = Math.max(12, Math.min(28, 8 + Math.sqrt(Math.max(level1.length, 1)) * 3.4));

level1.forEach((id, i) => {
  const angle = (i / Math.max(level1.length, 1)) * Math.PI * 2 - Math.PI / 2;
  positions.set(id, {
    x: Math.cos(angle) * R1,
    y: Math.sin(i * 0.7) * 0.25,
    z: Math.sin(angle) * R1,
  });
});

function layoutSubtree(parentId, depth) {
  const kids = children.get(parentId) || [];
  const parentPos = positions.get(parentId);
  if (!parentPos || !kids.length) return;

  // Direção "para fora" a partir da raiz (eixo do ramo)
  const outward = {
    x: parentPos.x,
    y: parentPos.y * 0.35,
    z: parentPos.z,
  };
  let olen = Math.hypot(outward.x, outward.y, outward.z);
  if (olen < 0.001) {
    outward.x = 0;
    outward.y = 0;
    outward.z = 1;
    olen = 1;
  }
  outward.x /= olen;
  outward.y /= olen;
  outward.z /= olen;

  // Base ortonormal no plano perpendicular ao ramo
  let ax = Math.abs(outward.x) < 0.9 ? 1 : 0;
  let ay = Math.abs(outward.x) < 0.9 ? 0 : 1;
  let az = 0;
  // u = outward x arbitrary
  let ux = outward.y * az - outward.z * ay;
  let uy = outward.z * ax - outward.x * az;
  let uz = outward.x * ay - outward.y * ax;
  let ul = Math.hypot(ux, uy, uz) || 1;
  ux /= ul;
  uy /= ul;
  uz /= ul;
  // v = outward x u
  let vx = outward.y * uz - outward.z * uy;
  let vy = outward.z * ux - outward.x * uz;
  let vz = outward.x * uy - outward.y * ux;

  const baseR = depth === 1 ? 7.5 : depth === 2 ? 5.4 : depth === 3 ? 4.0 : 3.2;
  const orbit = baseR + Math.min(Math.sqrt(kids.length) * 0.95, 8);
  const n = kids.length;
  // Leque aberto à frente do ramo (como o spread 2D), não esfera completa
  const spread = n <= 1 ? 0 : Math.min(Math.PI * 1.15, 0.55 + n * 0.38);
  const start = -spread / 2;
  const step = n <= 1 ? 0 : spread / (n - 1);

  kids.forEach((kidId, i) => {
    if (positions.has(kidId)) return;

```

```

    const a = n === 1 ? 0 : start + step * i;
    // leve elevação por índice para profundidade visual sem quebrar o raio
    const elev = ((i % 5) - 2) * 0.12;
    const cosA = Math.cos(a);
    const sinA = Math.sin(a);
    // ponto no cone: principalmente ao longo de outward + leque no plano u/v
    const along = orbit * 0.72;
    const side = orbit * 0.78;
    positions.set(kidId, {
      x: parentPos.x + outward.x * along + (ux * cosA + vx * sinA) * side,
      y: parentPos.y + outward.y * along + (uy * cosA + vy * sinA) * side + elev,
      z: parentPos.z + outward.z * along + (uz * cosA + vz * sinA) * side,
    });
    layoutSubtree(kidId, depth + 1);
  });
}

level1.forEach((id) => layoutSubtree(id, 1));

nodes.forEach((n) => {
  if (!positions.has(n.id)) {
    positions.set(n.id, clusterFallback(n, nodes));
  }
});

return nodes.map((n) => ({
  ...n,
  position: positions.get(n.id) || { x: 0, y: 0, z: 0 },
})));
}

function clusterFallback(node, nodes) {
  const cats = [...new Set(nodes.map((n) => n.category))];
  const idx = cats.indexOf(node.category);
  const a = (idx / Math.max(cats.length, 1)) * Math.PI * 2;
  const r = 24 + hash(node.id) * 8;
  return {
    x: Math.cos(a) * r,
    y: (hash(node.id + "y") - 0.5) * 6,
    z: Math.sin(a) * r,
  };
}

function hash(s) {
  let h = 0;
  for (let i = 0; i < s.length; i++) h = (h * 31 + s.charCodeAt(i)) >>> 0;
  return (h % 1000) / 1000;
}

export function getNeighbors(nodeId, links) {
  const set = new Set([nodeId]);
  (links || []).forEach((l) => {
    if (l.source === nodeId) set.add(l.target);
    if (l.target === nodeId) set.add(l.source);
  });
  return set;
}

export function getProjectSlug() {
  const p = new URLSearchParams(window.location.search);
  return p.get("project") || "professional-scanner";
}

export function countConnections(nodeId, links) {
  return (links || []).filter((l) => l.source === nodeId || l.target === nodeId).length;
}

export function countDependencies(nodeId, links) {
  return (links || []).filter((l) => l.target === nodeId).length;
}

```

universe/src/components/GalaxyGraph.jsx

```

import { Suspense, useMemo, useState, useCallback } from 'react';
import { Canvas } from '@react-three/fiber';
import { OrbitControls } from '@react-three/drei';
import { StarField, GridFloor, NodeSphere, ConnectionBeam } from '../SceneObjects';
import { CameraFocus, ResetCamera } from '../CameraFocus';

```

```

import { computeLayout, getNeighbors } from '../utils/layout';

function Scene({
  graph,
  selectedId,
  hoveredId,
  onSelect,
  onOpenCode,
  onHover,
  showLabels,
  showLinks,
  showParticles,
  showAnimations,
  activeCategories,
  resetCam,
  paused,
}) {
  const laidOut = useMemo(() => computeLayout(graph), [graph]);
  const posMap = useMemo(() => new Map(laidOut.map((n) => [n.id, n.position])), [laidOut]);
  const sizeMap = useMemo(
    () => new Map(laidOut.map((n) => [n.id, Math.max(0.55, n.size * (n.layer === 'root' ? 1.1 : 1))])),
    [laidOut]
  );
  const links = graph.links || [];

  const focusNode = useMemo(() => {
    if (!selectedId) return null;
    const n = laidOut.find((x) => x.id === selectedId);
    return n ? n.position : null;
  }, [selectedId, laidOut]);

  const neighborSet = useMemo(
    () => (selectedId || hoveredId ? getNeighbors(selectedId || hoveredId, links) : null),
    [selectedId, hoveredId, links]
  );

  const hasFocus = Boolean(selectedId || hoveredId);

  return (
    <>
    <color attach="background" args={['#010409']} />
    <fog attach="fog" args={['#010409', 50, 150]} />
    <ambientLight intensity={0.35} />
    <directionalLight position={[30, 40, 20]} intensity={0.6} color="#93c5fd" />
    <pointLight position={[-30, 15, -25]} intensity={0.55} color="#818cf8" />
    <StarField />
    <GridFloor />
    <ResetCamera trigger={resetCam} />
    <CameraFocus target={focusNode} offset={[0, 4, 20]} />

    {showLinks &&
      links.map((link, i) => {
        const connected = neighborSet?.has(link.source) && neighborSet.has(link.target);
        const active = Boolean(showParticles && connected);
        const dimmed = hasFocus && !connected;
        return (
          <ConnectionBeam
            key={` ${link.source}-${link.target}-${i}`}
            link={link}
            posMap={posMap}
            sizeMap={sizeMap}
            active={active || (connected && !hasFocus)}
            dimmed={dimmed}
          />
        );
      })
    }

    {laidOut.map((node) => {
      if (!activeCategories.has(node.category)) return null;
      const selected = node.id === selectedId;
      const hovered = node.id === hoveredId;
      const highlighted = neighborSet?.has(node.id) ?? false;
      const dimmed = hasFocus && !highlighted;
      // Com foco: só labels dos vizinhos (menos caos). Sem foco: root/module + arquivos.
      const showLabel =
        showLabels &&
        (selected ||
          hovered ||
          node.layer === 'root' ||

```

```

        node.layer === 'module' ||
        (!hasFocus && node.layer === 'file') ||
        (hasFocus && highlighted));

    return (
      <NodeSphere
        key={node.id}
        node={node}
        selected={selected}
        hovered={hovered}
        highlighted={highlighted}
        dimmed={dimmed}
        showLabel={showLabel}
        onClick={onSelect}
        onDoubleClick={(n) => {
          if (n.layer === 'root') return;
          onOpenCode(n);
        }}
        onPointerOver={onHover}
        onPointerOut={() => onHover(null)}
      />
    );
  }}}

  <OrbitControls
    enableDamping
    dampingFactor={0.05}
    minDistance={6}
    maxDistance={110}
    maxPolarAngle={Math.PI * 0.49}
    autoRotate={showAnimations && !hasFocus && !paused}
    autoRotateSpeed={0.28}
  />
</>
);
}

export default function GalaxyGraph(props) {
  const [hoveredId, setHoveredId] = useState(null);
  const handleHover = useCallback((node) => setHoveredId(node?.id ?? null), []);

  return (
    <Canvas
      camera={{ position: [0, 12, 48], fov: 50 }}
      gl={{ antialias: true, alpha: false, powerPreference: 'high-performance' }}
      dpr={[1, 1.5]}
      frameLoop={props.paused ? 'never' : 'always'}
    >
      <Suspense fallback={null}>
        <Scene {...props} hoveredId={hoveredId} onHover={handleHover} />
      </Suspense>
    </Canvas>
  );
}

```

universe/src/components/SceneObjects.jsx

```

import { useRef, useMemo, useEffect } from 'react';
import { useFrame } from '@react-three/fiber';
import { Billboard } from '@react-three/drei';
import * as THREE from 'three';

export function StarField({ count = 5000 }) {
  const ref = useRef();
  const positions = useMemo(() => {
    const pos = new Float32Array(count * 3);
    for (let i = 0; i < count; i++) {
      const r = 90 + Math.random() * 140;
      const theta = Math.random() * Math.PI * 2;
      const phi = Math.acos(2 * Math.random() - 1);
      pos[i * 3] = r * Math.sin(phi) * Math.cos(theta);
      pos[i * 3 + 1] = r * Math.sin(phi) * Math.sin(theta);
      pos[i * 3 + 2] = r * Math.cos(phi);
    }
    return pos;
  }, [count]);

  useFrame(({ clock }) => {

```

```

    if (ref.current) ref.current.rotation.y = clock.elapsedTime * 0.004;
  });

  return (
    <points ref={ref}>
      <bufferGeometry>
        <bufferAttribute attach="attributes-position" count={count} array={positions} itemSize={3} />
      </bufferGeometry>
      <pointsMaterial size={0.4} color="#a5d8ff" transparent opacity={0.85} sizeAttenuation depthWrite={false} />
    </points>
  );
}

export function GridFloor() {
  return (
    <group position={[0, -14, 0]}>
      <gridHelper args=[[160, 50, '#1d4ed8', '#0f172a']] />
      <mesh rotation=[[-Math.PI / 2, 0, 0]] position={[0, -0.05, 0]}>
        <planeGeometry args=[[160, 160]] />
        <meshBasicMaterial color="#020617" transparent opacity={0.55} />
      </mesh>
    </group>
  );
}

/** Textura 2D no WebGL – mesmo sistema de coordenadas das arestas (sem Html/CSS). */
function makeLabelTexture(text, borderColor, selected) {
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');
  const padX = 14;
  const padY = 8;
  const font = selected ? '600 28px JetBrains Mono, Consolas, monospace' : '500 24px JetBrains Mono, Consolas, monospace';
  ctx.font = font;
  const label = text.length > 28 ? `${text.slice(0, 27)}...` : text;
  const tw = Math.ceil(ctx.measureText(label).width);
  const w = tw + padX * 2;
  const h = 36 + padY;
  canvas.width = w;
  canvas.height = h;

  // fundo
  ctx.fillStyle = selected ? 'rgba(15, 23, 42, 0.95)' : 'rgba(2, 6, 23, 0.88)';
  roundRect(ctx, 0.5, 0.5, w - 1, h - 1, 6);
  ctx.fill();
  // borda
  ctx.strokeStyle = borderColor || '#38bdf8';
  ctx.lineWidth = selected ? 2.5 : 1.5;
  roundRect(ctx, 1, 1, w - 2, h - 2, 6);
  ctx.stroke();
  // texto
  ctx.font = font;
  ctx.fillStyle = '#f8fafc';
  ctx.textBaseline = 'middle';
  ctx.fillText(label, padX, h / 2);

  const tex = new THREE.CanvasTexture(canvas);
  tex.colorSpace = THREE.SRGBColorSpace;
  tex.needsUpdate = true;
  const aspect = w / h;
  return { tex, aspect, worldH: selected ? 1.15 : 0.95 };
}

function roundRect(ctx, x, y, w, h, r) {
  ctx.beginPath();
  ctx.moveTo(x + r, y);
  ctx.arcTo(x + w, y, x + w, y + h, r);
  ctx.arcTo(x + w, y + h, x, y + h, r);
  ctx.arcTo(x, y + h, x, y, r);
  ctx.arcTo(x, y, x + w, y, r);
  ctx.closePath();
}

/**
 * O NÓ é o cartão Billboard no centro exato (onde o raio chega).
 * Sem Html – nada de projeção CSS desalinhada da cena WebGL.
 */
export function NodeSphere({
  node,
  selected,

```

```

hovered,
highlighted,
dimmed,
showLabel,
onClick,
onDoubleClick,
onPointerOver,
onPointerOut,
}) {
  const groupRef = useRef();
  const core = Math.max(0.28, (node.size || 0.85) * 0.32);
  const color = useMemo(() => new THREE.Color(node.color || '#38bdf8'), [node.color]);
  const opacity = dimmed ? 0.15 : 1;

  const labelGfx = useMemo(() => {
    if (!showLabel) return null;
    return makeLabelText(node.label || node.id, node.color, selected || hovered);
  }, [showLabel, node.label, node.id, node.color, selected, hovered]);

  useEffect(() => () => {
    labelGfx?.tex?.dispose();
  }, [labelGfx]);

  useFrame(() => {
    if (!groupRef.current) return;
    const s = selected ? 1.12 : hovered ? 1.06 : 1;
    groupRef.current.scale.lerp(new THREE.Vector3(s, s, s), 0.15);
  });

  const labelW = labelGfx ? labelGfx.worldH * labelGfx.aspect : 1;
  const labelH = labelGfx ? labelGfx.worldH : 1;

  return (
    <group ref={groupRef} position={[node.position.x, node.position.y, node.position.z]}>
      </* Núcleo – ponto exato do endpoint do raio */>
      <mesh
        onClick={(e) => { e.stopPropagation(); onClick(node); }}
        onDoubleClick={(e) => {
          e.stopPropagation();
          if (onDoubleClick) onDoubleClick(node);
        }}
        onPointerOver={(e) => { e.stopPropagation(); onPointerOver(node); document.body.style.cursor = 'pointer'; }}
        onPointerOut={() => { onPointerOut(); document.body.style.cursor = 'default'; }}
      >
        <sphereGeometry args={[core, 20, 20]} />
        <meshStandardMaterial
          color={color}
          emissive={color}
          emissiveIntensity={selected ? 2.2 : hovered ? 1.6 : highlighted ? 1.1 : 0.7}
          transparent
          opacity={opacity}
          roughness={0.25}
          metalness={0.7}
        />
      </mesh>

      {(selected || hovered || node.layer === 'root') && (
        <mesh scale={[core * 2.4, core * 2.4, core * 2.4]}>
          <sphereGeometry args={[1, 12, 12]} />
          <meshBasicMaterial color={node.color} transparent opacity={0.12} depthWrite={false} />
        </mesh>
      )}
    </group>

    </* Cartão no MESMO ponto (Billboard) – centro = âncora do raio */>
    {labelGfx && (
      <Billboard follow lockX={false} lockY={false} lockZ={false}>
        <mesh
          position={[0, 0, core + 0.02]}
          renderOrder={2}
          onClick={(e) => { e.stopPropagation(); onClick(node); }}
          onDoubleClick={(e) => {
            e.stopPropagation();
            if (onDoubleClick) onDoubleClick(node);
          }}
          onPointerOver={(e) => { e.stopPropagation(); onPointerOver(node); document.body.style.cursor = 'pointer'; }}
          onPointerOut={() => { onPointerOut(); document.body.style.cursor = 'default'; }}
        >
          <planeGeometry args={[labelW, labelH]} />
          <meshBasicMaterial

```

```

        map={labelGfx.tex}
        transparent
        opacity={opacity}
        depthTest
        depthWrite={false}
        side={THREE.DoubleSide}
      />
    </mesh>
  </Billboard>
)}
</group>
);
}

/** Aresta reta centro->centro (mesmo origin do Billboard). */
export function ConnectionBeam({ link, posMap, sizeMap, active, dimmed }) {
  const { line } = useMemo(() => {
    const start = posMap.get(link.source);
    const end = posMap.get(link.target);
    if (!start || !end) return { line: null };

    const s0 = new THREE.Vector3(start.x, start.y, start.z);
    const e0 = new THREE.Vector3(end.x, end.y, end.z);
    const dir = e0.clone().sub(s0);
    const dist = dir.length();
    if (dist < 0.01) return { line: null };
    dir.multiplyScalar(1 / dist);

    // Encosta no núcleo (esfera pequena), não "morde" o cartão
    const rs = Math.max(0.22, (sizeMap?.get(link.source) || 0.85) * 0.32);
    const re = Math.max(0.22, (sizeMap?.get(link.target) || 0.85) * 0.32);
    const s = s0.clone().addScaledVector(dir, rs);
    const e = e0.clone().addScaledVector(dir, -re);

    const geo = new THREE.BufferGeometry();
    geo.setAttribute(
      'position',
      new THREE.BufferAttribute(new Float32Array([s.x, s.y, s.z, e.x, e.y, e.z]), 3)
    );
    const mat = new THREE.LineBasicMaterial({
      color: active ? '#7dd3fc' : dimmed ? '#1e293b' : '#94a3b8',
      transparent: true,
      opacity: active ? 1 : dimmed ? 0.07 : 0.65,
      depthWrite: false,
    });
    return { line: new THREE.Line(geo, mat) };
  }, [link.source, link.target, posMap, sizeMap, active, dimmed]);

  useEffect(() => () => {
    if (!line) return;
    line.geometry.dispose();
    line.material.dispose();
  }, [line]);

  if (!line) return null;
  return <primitive object={line} />;
}

```